Progettazione Software

- **La progettazione è la fase in cui viene decisa e specificata la struttura del sistema.**
  - Lo strumento fondamentale è la divisione del sistema in sottosistemi (divide et impera)
    - I sottosistemi sono chiamati moduli o componenti.
  - La struttura del sistema è chiamata architettura.
    - Architettura software
- **Mira a trasformare un problema in una soluzione.**
  - La progettazione si concentra sul passaggio dai requisiti al design.
  - I requisiti definiscono il problema in termini di COSA deve essere fatto.
  - Il design propone una soluzione di COME realizzare ciò che è richiesto.
- **Si sviluppa per raffinamento**
  - ossia passando attraverso diverse fasi che definiscono in maniera sempre più dettagliata il prodotto finale
- **Si effettua la progettazione architetturale e dei singoli componenti**
  - La progettazione architetturale mappa i requisiti del sistema su componenti o sottosistemi, definendo una visione generale e strutturata dell'intero sistema.
  - La progettazione dei singoli componenti si concentra sui dettagli specifici di ogni componente e sottosistema, stabilendo in modo più preciso la soluzione da adottare.
- **Durante la progettazione, vengono fatte anche scelte tecnologiche che possono influire sul prodotto finale.**
  - Il progetto può essere indipendente dalla piattaforma o orientato a una specifica piattaforma (progetto platform-specific) come .Net o J2EE.
  - La scelta di tecnologie specifiche può essere compatibile solo con la piattaforma prescelta.

Presented with xmind

Architettura software

- **Si divide in tre livelli distinti**
  - Architettura generale hardware + software
    - Riguarda hardware e software
      - Tipo di calcolatori
      - Tipo di rete
      - Dispositivi utilizzati
  - Architettura software di base
    - Riguarda il software necessario per far funzionare il sistema
      - Sistema operativo
      - Protocolli di rete
      - Linguaggi di programmazione utilizzati
  - Architettura dell'applicazione
    - riguarda la struttura dell'applicazione da realizzare
      - Divisione in moduli
      - Interazione tra moduli
- **Esistono diversi stili architetturali**
  - Definiti dai tipi di componenti base utilizzati
    - Classi
    - Filtri
    - Strati o sottosistemi
  - Definiti dai tipi di connettori utilizzati per collegare i componenti
    - Chiamate di funzione
    - "Pipes"
    - Notifiche di eventi
    - Chiamate HTTP
  - Definiti dal tipo di controllo utilizzato per attivare i moduli
    - Modo in cui i moduli sono attivati
    - Chi attiva i moduli
- **Componenti base di un software**
  - Sistema operativo
  - Protocollo di rete
  - DBMS
  - Server Internet
  - PDE (linguaggio + IDE + framework)
  - Modello dei componenti
  - Web services
  - dApp (sistema decentralizzato, spesso basato su una blockchain)
- **Stili architetturali**
  - Architettura della singola applicazione
    - Divisa in due classi
      - Modelli di scomposizione (Architettura generale del sistema)
        - Si concentra sulla scomposizione dell' applicazione in moduli
        - Modelli
          - Architettura a strati
          - Modello Client-Server
            - Architettura basata su Internet
              - SOA (Service Oriented Architecture)
                - Microservizi
                - Modello Broadcast
            - Two-Tier e Three-Tier
              - Two-Tier
                - Two-Tier con fat client
                - Il client che richiede il servizio e interagisce con l'utente, e il server, che fornisce il servizio.
                - Il client implementa la logica applicativa e la GUI, mentre il server è un DBMS
                - Two-Tier con thin client
                - il client è un dispositivo a bassa potenza che comunica con il server attraverso una connessione di rete, interagisce con l'utente tramite un'interfaccia minimale.
                - Il client possiede solo una GUI mentre nel server è presente il DBMS e la logica applicativa
              - Architettura Three-Tier
            - Enterprise Service Bus (ESB)
            - Cloud Computing
          - Parallelismo
          - Peer-to-peer (P2P)
            - Architetture Decentralizzate (dApp)
          - Modello Embedded
      - Modelli di controllo
        - Si concentra sul controllo dell'elaborazione, ovvero su come il flusso di dati e di controllo attraversa l'applicazione.
        - Modelli
          - Controllo centralizzato
            - Un singolo modulo o controllore ha il compito di controllare gli altri moduli passivi, che rispondono solo al controllore.
            - Ci sono due tipi di controllo centralizzato:
              - Modello call and return
              - Manager Model
          - Controllo a flusso di dati (Dataflow)
            - Utile in situazioni in cui la disponibilità dei dati determina il controllo
            - definisce la sequenza in cui i dati vengono elaborati da una serie di componenti software e il modo in cui i dati fluiscono attraverso di essi.
            - Il modello assume che ogni componente software sia responsabile solo di elaborare i dati che riceve in input e di produrre dati in output, senza effettuare alcuna azione sul sistema al di fuori del proprio ambito.
            - Utilizzato in contesti in cui i dati sono dinamici e cambiano frequentemente, come ad esempio nei sistemi di elaborazione in tempo reale
            - Modelli
              - Sequenziale batch
              - Pipes and filter
              - Controllo ad eventi
              - Modello centrato sui dati
              - Modello a "Interrupt"
              - Modello a coda degli eventi
          - Modello ad oggetti
          - Modello a Plug-In
    - Architetture miste
      - Combina più stili architetturali per creare un' architettura eterogenea
      - I sistemi software raramente seguono un solo stile architetturale

Presented with xmind

Architettura a strati

- **Organizza il sistema in vari strati**
  - Ogni strato fornisce servizi allo strato sopra di esso
  - Ogni strato dipende solo dallo strato sotto di esso
  - Ogni strato raccoglie i servizi a un dato livello di astrazione
    - Strati più interni effettuano operazioni vicine all'hardware
    - Strati più esterni forniscono servizi di livello sempre più elevato e astratto
- **Utilizza due tipi di componenti**
  - Gruppo di funzionalità implementano un' astrazione a un dato livello della gerarchia
    - Espongono un'interfaccia verso lo strato superiore con l'elenco dei servizi forniti
  - Connettori: protocolli che definiscono come gli strati interagiscono e la modalità di chiamata dei servizi dell'interfaccia
- **Vantaggi:**
  - Separazione in livelli di astrazione
  - Basso accoppiamento
  - Se cambia l'interfaccia di uno strato, solo lo strato superiore ne è influenzato
  - Supporta il riuso
- **Svantaggi:**
  - Spesso è artificiale strutturare il sistema in questo modo
  - Può risultare inefficiente e spesso questo porta a "scorciatoie" (shortcut) si salta qualche livello

Presented with xmind

Modello Client-Server

- **È un'architettura distribuita**
- **Elaborazioni eseguite da più processori**
  - Scambio di dati tramite connettori di flusso di dati
    - Tipicamente inviati tramite una rete.
- **Composto da due tipi di componenti: server e client**
  - I server offrono servizi di elaborazione
    - gestione dei dati
    - la stampa
    - la connessione a Internet
  - I client utilizzano i servizi offerti dai server
    - sono generalmente costituiti da PC o workstation connessi in rete, su cui girano le applicazioni utente.
    - Il client conosce i servizi disponibili dei server, mentre i server non conoscono i client.
  - Distinzione tra client e server è logica e non fisica
    - non è necessario che il server sia fisicamente separato dal client.
- **Esistono diversi modelli di client in base alla "intelligenza"**
  - Thin client
    - quasi tutti i calcoli sono effettuati dai server
    - i sistemi utilizzati dall'utente per collegarsi hanno un'intelligenza molto limitata;
  - Modello di intelligenza bilanciata
    - sia il client che i server effettuano calcoli.
    - Il client effettua tutti i calcoli locali possibili, mentre il server effettua tutte le elaborazioni che è ragionevole effettuare presso di esso
  - Fat client
    - la maggioranza delle elaborazioni sono effettuate sul client, e il server è un gestore di dati passivo.
    - Talvolta il client esegue una copia locale dei dati.
- **Vantaggi del client-server**
  - Sistema distribuito e modulare
  - Utilizzo efficiente delle reti
  - Possibilità di utilizzare hardware meno costoso
  - Tolleranza ai guasti (se un server si blocca, il sistema continua a funzionare)
  - Facilità di cambiare la struttura interna di un server senza impatto sul resto del sistema
- **Svantaggi del client-server**
  - Architettura complessa
  - Integrazione dei client coi server complessa
  - Costi di programmazione e gestione elevati
  - Problemi di scalabilità in caso di aggiunta di nuovi server e client (capacità del sistema di gestire un aumento del carico di lavoro o del numero di utenti senza subire una riduzione delle prestazioni o l'insorgere di problemi di funzionamento)
- **Composte da tre componenti:**
  - Magazzino per i dati permanenti e la loro gestione (repository)
  - Logica applicativa che elabora i dati
  - Interfaccia utente che presenta i dati e riceve le richieste degli utenti

Presented with xmind

Architettura basata su Internet

- Caratteristica distintiva: utilizzo della rete e delle comunicazioni basate su TCP-IP
- È un modello moderno per realizzare applicazioni client-server.
  - Client: browser web con eventuali plug-in appositi
  - Server: applicazioni interfacciate alla rete dal lato server, capaci di accettare connessioni
- Informazioni scambiate in formato XML o JSON
- Vantaggi
  - Utilizzo di standard aperti e di basso costo
  - Esistenza di applicazioni open source per la memorizzazione e lo scambio dei dati
  - Facilità di scalabilità
  - Formato per lo scambio dati basato sui caratteri
- Svantaggi
  - Velocità di scambio dei dati può essere critica in alcuni casi
  - Problemi di sicurezza per reti aperte all' esterno
  - Sviluppo di applicazioni complesso

Presented with xmind

Architettura Three-Tier

- **Si compone di tre strati distinti.**
  - Interfaccia utente:
    - Thin client, con i calcoli eseguiti lato server
    - Fat client, con i calcoli eseguiti lato client
  - Logica applicativa:
    - Eseguita su uno strato separato noto come Web Server
  - Gestione del database:
    - Può essere distribuita su più server
- **Vantaggi dell'architettura Three-Tier:**
  - Separazione dei tre componenti dell' applicazione
  - Gestione separata e più efficiente
  - Applicazioni modulari e facilmente manutenibili
- **Svantaggi dell'architettura Three-Tier:**
  - Implementazione più complessa rispetto al modello Two-Tier

Presented with xmind

SOA (Service Oriented Architecture)

- Definizione e disponibilità di servizi tramite protocolli HTTP o HTTPS
- Orchestrazione dei servizi per creare un'applicazione complessa
  - I client possono essere applicazioni connesse alla rete o pagine HTML visualizzate da un browser.
- Scambio dei dati in formato XML o JSON
- Architettura flessibile e scalabile per la distribuzione di servizi su Internet
- Componenti dell'architettura SOA
  - Service Broker
    - Registrazione e scoperta dei servizi
    - Registro dei servizi disponibili
    - Inoltro delle richieste di servizio ai Service Provider appropriati
  - Service Consumer
    - Utilizzo dei servizi offerti dall'architettura SOA
    - Accesso al Service Broker per trovare il servizio di cui ha bisogno
    - Gestione delle transazioni di servizio e sicurezza dell'interazione
  - Service Provider
    - Fornitura dei servizi offerti dall'architettura SOA
- Standard SOA stabiliti da W3C e OASIS
  - SOAP (Simple Object Access Protocol)
    - Definizione dei messaggi (richieste di servizio e risposta)
  - WSDL (Web Services Description Language)
    - Descrizione dell'interfaccia di un servizio Web: operazioni, protocollo di comunicazione, formato dei messaggi, vincoli, URI
  - UDDI (Universal Description Discovery and Integration)
    - Pubblicazione e ricerca dei Web Services disponibili su Internet (poco utilizzato)

Presented with xmind

Enterprise Service Bus (ESB)

- **Progettato per facilitare le comunicazioni tra applicazioni Client-Server diverse.**
  - L'ESB opera su una rete locale o su Internet e funge da intermediario tra le applicazioni, fornendo una serie di servizi.
- **Servizi forniti dall'ESB**
  - Unico punto di accesso per lo scambio di dati tra applicazioni
  - Controllo di autenticazione per garantire la sicurezza dell'accesso
  - Gestione delle autorizzazioni e dei livelli di accesso
  - Standardizzazione delle interfacce e trasformazioni dei dati
    - In modo da garantire la compatibilità tra le diverse applicazioni
  - Gestione dell'architettura SOA
- **Vantaggi dell'ESB**
  - Facilita l'integrazione delle applicazioni
  - Configurazione invece di codifica
  - Fornisce servizi di routing, interfaccia e sicurezza
  - Scalabile per grandi organizzazioni
  - Flessibilità in caso di cambiamenti
- **Svantaggi dell'ESB**
  - Sistema accoppiato all'ESB
  - Sovraccarichi e problemi di prestazioni
  - Possibile complessità

Presented with xmind

Microservizi

- Nasce dalla SOA, ovvero l'architettura orientata ai servizi
- L'applicazione viene vista come un insieme di servizi
  - Ogni servizio è un processo in esecuzione
  - I servizi comunicano con meccanismi " leggeri" come le API di risorse HTTP
  - I servizi implementano funzioni di business
  - I servizi possono essere installati indipendentemente, anche su macchine diverse
    - Per questo la gestione e la manutenzione dell' architettura è più flessibile e scalabile
  - I servizi rispondono a messaggi esterni inviati attraverso le API o messaggi inviati da altri servizi
  - I servizi possono essere scritti in diversi linguaggi di programmazione e utilizzare diverse tecnologie di archiviazione dei dati.
- La gestione centralizzata dei servizi è mantenuta al minimo
  - Questo permette una maggiore autonomia e indipendenza tra i servizi

Presented with xmind

Modello Broadcast

- **Caratteristiche**
  - Evoluzione dell'architettura a microservizi
  - Collegamento diretto o indiretto tra i componenti di un sistema client-server
  - Messaggio inviato a tutti i componenti o server anziché a uno specifico
  - Messaggio può essere un annuncio di evento o la richiesta di un servizio
  - Componenti in ascolto
  - Componenti destinatari o interessati all'evento eseguono la procedura associata ad esso
- **Vantaggi**
  - Maggiore flessibilità e scalabilità nell'architettura a microservizi
  - Semplice aggiunta/rimozione/sostituzione di componenti
  - Possibilità di riuso di componenti in diversi sistemi
- **Svantaggi**
  - Componenti non sanno se e quando la richiesta verrà gestita
  - Complessità nello scambio dei dati:
    - Dati passati con la richiesta o tramite repository
    - Possibili problemi di prestazioni
  - Difficoltà di implementazione e verifica

Presented with xmind

Cloud Computing

Descrizione

- Modello di servizio informatico in cui i server sono allocati su Internet da una società che fornisce servizi Cloud
- Fornitore si occupa di tutti i servizi di gestione, ridondanza, backup e scalabilità
- Ideale per le applicazioni che devono essere fruite su Web, come e-commerce, social network e servizi aziendali distribuiti

Tre modelli di Cloud Computing:

- Infrastructure as a Service (IaaS)
  - L'hardware come CPU, RAM e dischi sono forniti su Cloud computer e storage remoti e utilizzabili come se fossero locali.
- Platform as a Service (PaaS)
  - Su Cloud sono presenti tutti i componenti di un ambiente di sviluppo
  - Questi componenti permettono di sviluppare, testare, implementare e gestire le applicazioni aziendali senza i costi e la complessità associati all'acquisto, configurazione, ottimizzazione e gestione dell'hardware e del software di base.
- Software as a Service (SaaS)
  - Su Cloud sono presenti applicazioni specifiche che forniscono servizi.

Vantaggi del Cloud Computing:

- Abbattimento dei costi fissi, si paga a consumo
- Maggiore flessibilità e aggiornamento costante dell'hardware e del software di base
- Scalabilità
- Accesso 24/7 da qualunque terminale Internet
- Sicurezza e backup inclusi

Svantaggi del Cloud Computing:

- Dipendenza da Internet
- Problemi di privacy e sicurezza
- Dipendenza dal fornitore (lock-in)
  - L'utente diventa dipendente dal fornitore di servizi cloud
  - Non è facile spostarsi su un'altra piattaforma o provider
  - Limita le scelte dell'utente
  - Potrebbe aumentare i costi a lungo termine
- Perdita del controllo sui propri servizi IT
  - I servizi IT sono gestiti dal provider di cloud
  - L'utente perde il controllo sui propri servizi IT
  - Limita l'abilità dell'utente di personalizzare i propri servizi IT
  - Potrebbe comportare costi aggiuntivi per eventuali modifiche o personalizzazioni

Presented with xmind

Parallelismo

- Architetture per il supercalcolo
  - Utilizzano molti processori collegati tramite reti speciali
  - Processori di serie e una rete di connessione appositamente progettata
  - Alcuni casi, anche milioni di processori utilizzati
- Programmazione di supercalcolatori
  - Utilizzo di librerie e linguaggi speciali
  - Necessità di applicazioni scritte appositamente per sfruttare appieno la capacità di parallelismo

Presented with xmind

Peer-to-peer (P2P)

Caratteristiche

- Modello di rete informatica distribuito
- Ogni componente svolge il ruolo sia di client che di server
  - Ogni componente ha un Interfaccia che specifica i servizi offerti e quelli richiesti
  - I ruoli non sono definiti come nel modello client-server
- Peer comunicano tra loro richiedendo e offrendo servizi
- Distinzione tra i peer: i dati
  - Se un peer ha bisogno di un dato, lo chiede a un altro peer
- Scalabilità e tolleranza agli errori
  - Aggiunta di un nuovo peer aggiunge nuove capacità
  - I dati sono replicati su diversi peer
    - Garantendo una maggiore affidabilità.

Vantaggi del P2P

- Efficienza molto alta
  - Siccome tutti i nodi forniscono risorse
- Potenza del sistema cresce con il numero di componenti
  - Rende il sistema altamente scalabile
- Robustezza grazie alla replicazione dei dati su diversi peer
  - In quanto i dati sono replicati tra i vari peer e non esistono punti di fallimento del sistema.

Svantaggi del P2P

- Architettura complessa
- Controllo limitato sulle risorse
  - Poichè queste sono distribuite su diverse macchine.
- Richiesta di prestazioni dai peer
- Soggetto ad attacchi e problemi di sicurezza, come il Sybil attack

Presented with xmind

Architetture Decentralizzate (dApp)

Caratteristiche

- Sono applicazioni basate su una blockchain
  - La blockchain è un registro append-only delle transazioni accettate
  - La blockchain è una successione di blocchi che contengono il digest hash del blocco precedente e le transazioni valide registrate
  - Le blockchain di seconda generazione possono eseguire programmi chiamati Smart Contracts, che sono come dei calcolatori distribuiti
    - Gli Smart Contract sono programmi scritti in un linguaggio di alto livello (come Solidity in Ethereum) e tradotti in bytecode
      - Vengono eseguiti su tutti i nodi, e tutte le esecuzioni devono produrre lo stesso risultato.
      - Le chiamate di funzioni e le modifiche allo storage vengono registrate nella blockchain
      - Uno Smart Contract non può accedere a nessun dispositivo o direttamente alla rete
    - La blockchain è pubblica o permissioned (a invito)
      - Se è pubblica chiunque può gestire un nodo e partecipare alla rete.
        - Il consenso si ottiene attraverso il mining, il primo nodo che risolve un problema difficile incassa della criptovaluta.
        - La criptovaluta è associata a un indirizzo e può essere trasferita solo firmando una transazione con la chiave privata
      - Se la blockchain è permissioned (a invito), viene utilizzata per applicazioni "industriali", dove i nodi sono trusted.
- Utilizzano un sistema peer-to-peer (P2P)
- Ogni nodo mantiene una copia della blockchain
- I nodi utilizzano un meccanismo di consenso per decidere quale blocco accettare e aggiungere alla blockchain

Le dApp tipicamente interagiscono con sistemi Web più tradizionali che creano e inviano le transazioni

Vantaggi delle architetture decentralizzate:

- Decentralizzazione — Non esiste un'autorità centrale
- Immutabilità — La blockchain, una volta scritta, non si può alterare
- Sicurezza del possesso della criptovaluta o dell'identità di chi invia una transazione data dalla crittografia
- Trasparenza — Tutto è visibile, incluso il codice degli Smart Contract

Svantaggi delle architetture decentralizzate:

- Complessità dell'architettura
- Poche transazioni al secondo con blockchain pubbliche
- La blockchain ha dimensioni che tendono ad aumentare sempre
- Dati ed elaborazioni costano GAS, e sono quindi ridotti
  - Le transazioni devono essere confermate dalla rete attraverso il processo di mining, il che richiede una certa quantità di risorse computazionali e di energia.
  - Per motivi di sicurezza e per evitare abusi, queste risorse sono limitate e hanno un costo, chiamato GAS.
    - Questo può essere uno svantaggio per le dApp, poiché il costo del GAS può aumentare con il volume di transazioni e di elaborazioni richieste dalla dApp;
- Problemi con GDPR (per la trasparenza e immutabilità)
  - Legge europea
  - Obblighi dei fornitori di servizi online — Protezione dei dati personali degli utenti
  - Difficoltà nell'adempimento delle norme GDPR con blockchain
    - Trasparenza dei dati sulla blockchain
    - Immutabilità dei dati sulla blockchain
  - Difficoltà delle dApp nel garantire il diritto degli utenti di essere "dimenticati" — Rimozione dei dati personali dalla piattaforma

Presented with xmind

Modello Embedded

- **Software embedded**
  - Software eseguito all'interno di un dispositivo hardware dedicato
  - Esempi di dispositivi: microcontrollori, sistemi di controllo dell'automobile, dispositivi di rete, telefoni cellulari, elettrodomestici, sistemi di controllo del traffico aereo, sistemi di sicurezza per automobili, ecc.
  - Scopo: controllare le funzioni specifiche del dispositivo
- **Architettura del software embedded**
  - Dipende dalle specifiche dell'hardware e dalle esigenze dell'applicazione
  - Può essere progettato per girare direttamente sull'hardware o diviso in più componenti collegati tramite bus standard
- **Sviluppo del software embedded**
  - Spesso avviene su emulatori, cioè su ambienti simulati
    - Motivi: piattaforma reale potrebbe non esistere o essere molto costosa o difficile da utilizzare per il test e lo sviluppo del software
  - Problemi comuni: scarsità di risorse e di prestazioni, richiedono una progettazione attenta e ottimizzata del software.

Presented with xmind

Modello call and return

Caratteristiche

- Componenti: oggetti e funzioni (o subroutine)
- Connettori: chiamate di metodi o chiamate di funzione
- Gli oggetti incapsulano l'informazione e la rappresentazione interna è nascosta agli oggetti client.
- Linguaggi procedurali come il C utilizzano questa scomposizione in funzioni e sotto-funzioni
- Controllore rappresentato dal main()

Vantaggi

- Scomposizione chiara e ben nota
- Basata sull'analisi del funzionamento del sistema
  - ogni componente svolge una specifica funzione
  - Aiuta a mantenere il codice organizzato e facile da gestire
- Supporta lo sviluppo top-down
  - in cui si parte dalla definizione dei requisiti dell'applicazione e si procede con la scomposizione del sistema in moduli e componenti, in modo da sviluppare ogni componente separatamente.
  - permette di mantenere il focus su un solo modulo alla volta, semplificando il processo di sviluppo e riducendo i rischi di errori.

Svantaggi

- Tende a focalizzarsi su una funzione per volta
- Non dà la dovuta importanza ai dati
- Le funzioni perdono il contesto quando terminano
- Non adatto per il progetto di sistemi completi moderni con GUI, interfaccia Web, ecc.

Presented with xmind

Manager Model

Caratteristiche

- Modulo chiamato Manager controlla l'avvio, terminazione e coordinazione degli altri processi del sistema
  - Manager può essere una funzione, un oggetto o un sottosistema
- Utilizzato in sistemi di controllo senza vincoli di tempo critici
- Manager esegue un ciclo infinito per controllare lo stato dei sensori e dei moduli di input, eseguendo elaborazioni utilizzando il proprio stato

Presented with xmind

Sequenziale batch

Caratteristiche

- Il controllo del flusso di dati viene gestito attraverso un'elaborazione sequenziale
- I dati sono suddivisi in batch e inviati al sistema in modo sequenziale
- Ogni batch viene elaborato in modo autonomo e indipendente dagli altri batch
- I dati di output possono essere inviati al sistema successivo o all'utente dopo l' elaborazione di ogni batch
- Componenti del sistema includono il software di gestione dei batch, il software di elaborazione dei dati, e il software di gestione dell'output

Vantaggi

- capacità di elaborare grandi quantità di dati in modo efficiente
- elaborazione dei dati in modo indipendente
- può essere utilizzato per l'elaborazione di dati offline

Svantaggi:

- l'elaborazione dei dati non è in tempo reale
- può essere inefficiente nell'elaborazione di piccole quantità di dati
- complesso da implementare in sistemi multi-componente.

Presented with xmind

Pipes and filter

Caratteristiche

- Architettura software basata su una pipeline di filtri che elaborano i dati in modo sequenziale
- Ogni filtro riceve i dati in input, li elabora e produce in output un nuovo set di dati
- I filtri sono organizzati in modo da costituire una pipeline, dove i dati fluiscono da uno all' altro
- Esempio di applicazione: elaborazione di un file di testo

Vantaggi

- supporta il riuso delle trasformazioni
- intuitivo, permette di capire il funzionamento del sistema come composizione individuale dei filtri
- facile da aggiungere nuove trasformazioni
- abbastanza semplice da implementare
- supporta esecuzione concorrente

Svantaggi:

- non adatto per sistemi interattivi
- lavoro extra per parsing e unparsing dei dati nei vari filtri
- difficile accedere a dati globali
- la gestione degli errori è difficile

Presented with xmind

Controllo ad eventi

- **Caratteristiche**
  - modello in cui i componenti del sistema si registrano per eventi specifici e vengono invocati quando un evento accade, eseguendo l'operazione associata (callback).
  - Un evento può essere generato da un input esterno o da un'elaborazione interna e può essere associato a dei dati trasmessi al componente che lo gestisce
- **Vantaggi:**
  - Non richiede una conoscenza diretta dei componenti che gestiranno l'evento.
  - Supporta il riuso dei componenti.
    - I componenti possono essere aggiunti e rimpiazzati con effetti minimi sul resto del sistema.
- **Svantaggi:**
  - Perdita di controllo su quale componente processerà gli eventi e in che ordine saranno invocati.
  - Sistema difficile da testare poiché i risultati possono dipendere dal contesto e dall'ordine di invocazione.
- **Modello "broadcast" o Event based o Publish-Subscribe:**
  - È una forma specifica di controllo ad eventi.
  - Quando un componente annuncia un evento, lo invia a tutti gli altri componenti interessati e questi eseguono la procedura associata.

Presented with xmind

Modello centrato sui dati

Caratteristiche

- Utilizzo di un "magazzino" centrale dei dati chiamato Central Repository o Data Store.
- Tutti i componenti del sistema operano indipendentemente su di esso.
- Richieste di servizi e risposte al/dal Data Store sono gestite dai connettori.
- Utile quando molti utenti devono agire sugli stessi dati.
- Il Data Store di solito è implementato tramite un DBMS.

Vantaggi

- Efficienza
- Assenza di ridondanza dei dati
- Gestione centralizzata dei dati (sicurezza, backup)
- Le applicazioni non devono conoscersi.

Svantaggi

- Modello dei dati comune a tutte le applicazioni
- Evoluzione costosa del modello dei dati
- Problemi di carico in caso di accesso massiccio
- sensibile all'indisponibilità del Data Store o della rete.

Esempio

- gestione del magazzino, database in un server dedicato, applicazione su un Web server, accesso tramite browser web.
  - Ogni sessione crea un'istanza dell' applicazione che si collega al DB e presenta i dati all'utente, l'utente può creare, cancellare e modificare i dati

Presented with xmind

Modello a "Interrupt"

- **Segnale asincrono**
  - Indica il "bisogno di attenzione" da parte di:
    - Periferica che richiede un servizio
    - Evento per interrompere un processo
    - Richiesta particolare al sistema operativo da parte di un processo in esecuzione
- **Gestione a livello di CPU e di SO**
- **Essenziale per sistemi Real time con vincoli stretti**
- **Riguarda la progettazione/ programmazione a basso livello**
  - Sistemi operativi
  - Controllori di periferiche o controllori industriali

Presented with xmind

Modello a coda degli eventi

Caratteristiche

- Combinazione di modello Manager e gestione degli eventi
- Usato per gestire gli eventi del dominio applicativo
- Gli eventi sono oggetti con dati e metodo execute
  - Ogni evento ha tempo di elaborazione e priorità
  - Gli eventi vengono inseriti in una coda ordinata per tempo
- Controllore centrale estrae l'evento più recente o prioritario
  - Se le priorità sono le stesse, l'estrazione avviene in ordine di entrata o a caso.
  - L'evento viene elaborato con il metodo execute
  - L'elaborazione può generare nuovi eventi
- Sensori esterni possono generare eventi
- Esiste un evento di fine elaborazione
  - se eseguito, fa terminare il ciclo

Vantaggi

- Permette di gestire gli eventi in modo efficiente e flessibile
- Garantisce un'elaborazione rapida e prioritaria degli eventi importanti per il dominio applicativo.

Presented with xmind

Modello ad oggetti

- **Caratteristiche**
  - Unità di scomposizione: oggetto
    - Oggetto: insieme di dati, metodi e interfaccia esterna
  - Invio di un messaggio per chiamare un' operazione su un oggetto
  - Interfaccia esterna come unica conoscenza necessaria per utilizzare un oggetto
  - Oggetti simili sono descritti da una classe
  - Relazioni tra classi tramite ereditarietà
- **Vantaggi:**
  - Implementazione degli oggetti modificabile senza influenzare i clienti
    - Nozione di interfaccia
  - Gli oggetti possono riflettere entità del mondo reale
  - Linguaggi di implementazione object oriented diffusi e conosciuti
- **Svantaggi:**
  - Un oggetto deve conoscere l'identità di un altro per interagire
  - Modifica dell'interfaccia comporta la modifica di tutti gli oggetti che la usano
  - Alcuni linguaggi prevedono oggetti con granularità troppo fine

Presented with xmind

Modello a Plug-In

- **Plugin**
  - Componente software che aggiunge funzionalità ad un'applicazione esistente
  - Chiamato anche add-in, estensione o add-on
- **Caratteristiche**
  - L'applicazione deve essere predisposta ad accettare plug-in
    - Protocollo per la chiamata di servizi e scambio di dati coi plug-in
    - Possibilità di registrare, attivare e rendere disponibili nuovi plug-in all'utente
    - Possibilità di eliminare plugin non più richiesti
  - Il Plug-in utilizza i servizi messi a disposizione dall'applicazione tramite un' interfaccia nota
  - Espone un'interfaccia standard per essere riconosciuto e gestito dall'applicazione

Presented with xmind